

CS 305: Computer Networks

Fall 2025

Week 15 Wireless and Mobile Networks

Ming Tang

Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)

Chapter 7 outline

7.1 Introduction

Wireless

7.2 Wireless links, characteristics

- CDMA

7.3 IEEE 802.11 wireless LANs (“Wi-Fi”)

7.4 Cellular Internet access

- architecture
- standards (e.g., 3G, LTE)

Mobility

7.5 Principles: addressing and routing to mobile users

7.6 Mobile IP

7.7 Handling mobility in cellular networks

7.8 Mobility and higher-layer protocols

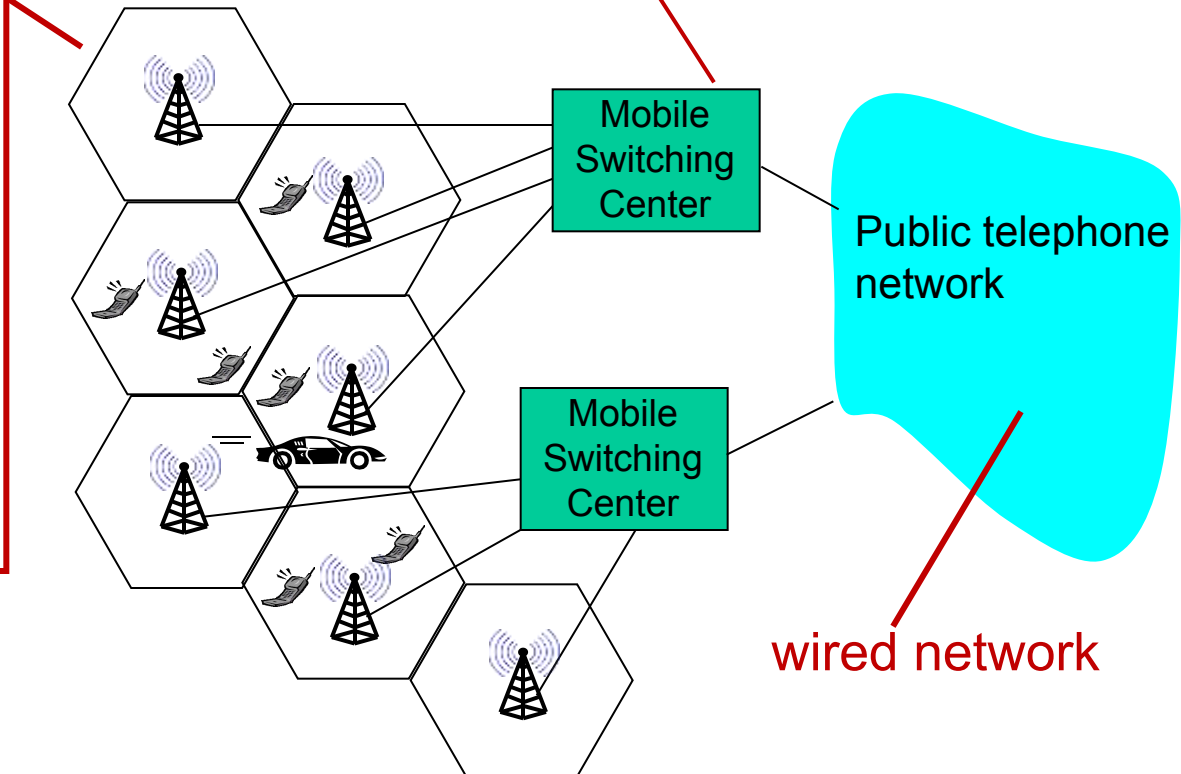
Components of cellular network architecture

cell

- ❖ covers geographical region
- ❖ *base station* (BS) analogous to 802.11 AP
- ❖ *mobile users* attach to network through BS
- ❖ *air-interface*: physical and link layer protocol between mobile and BS

MSC

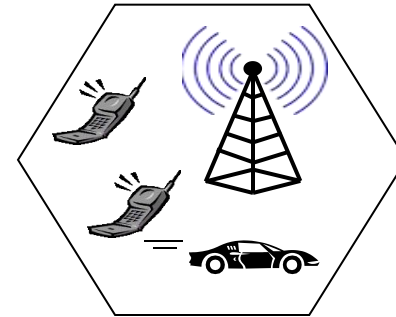
- ❖ connects cells to wired tel. net.
- ❖ manages call setup (more later!)
- ❖ handles mobility (more later!)



Cellular networks: the first hop

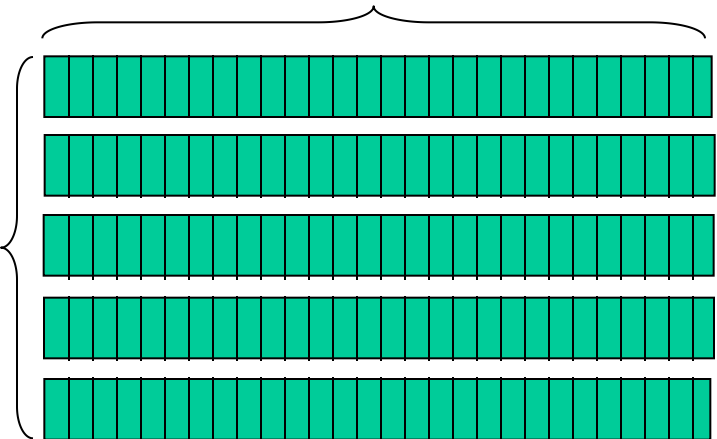
Two techniques for sharing mobile-to-BS radio spectrum

- **combined FDMA/TDMA:** divide spectrum in frequency channels, divide each channel into time slots
- **CDMA:** code division multiple access

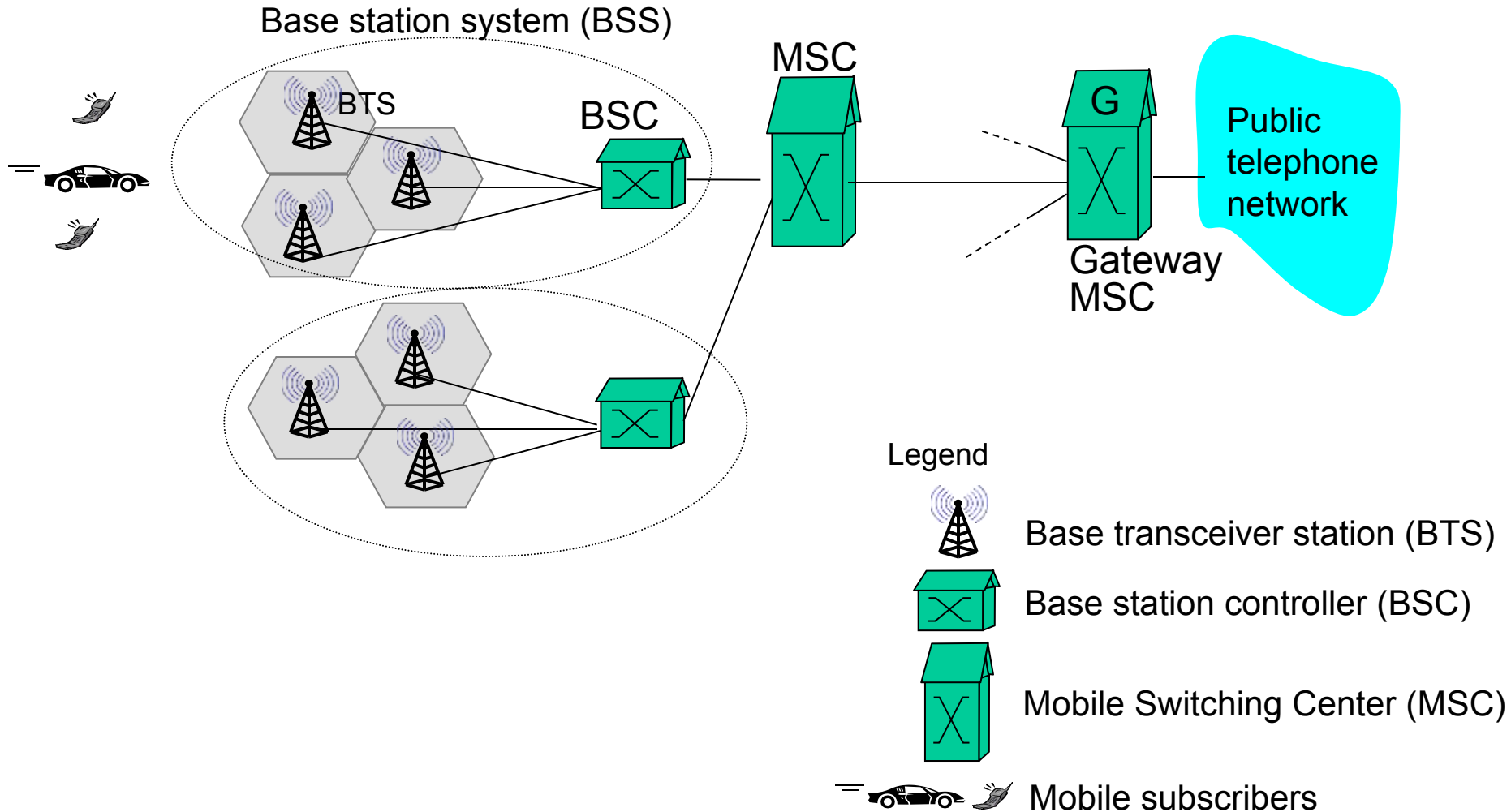


time slots

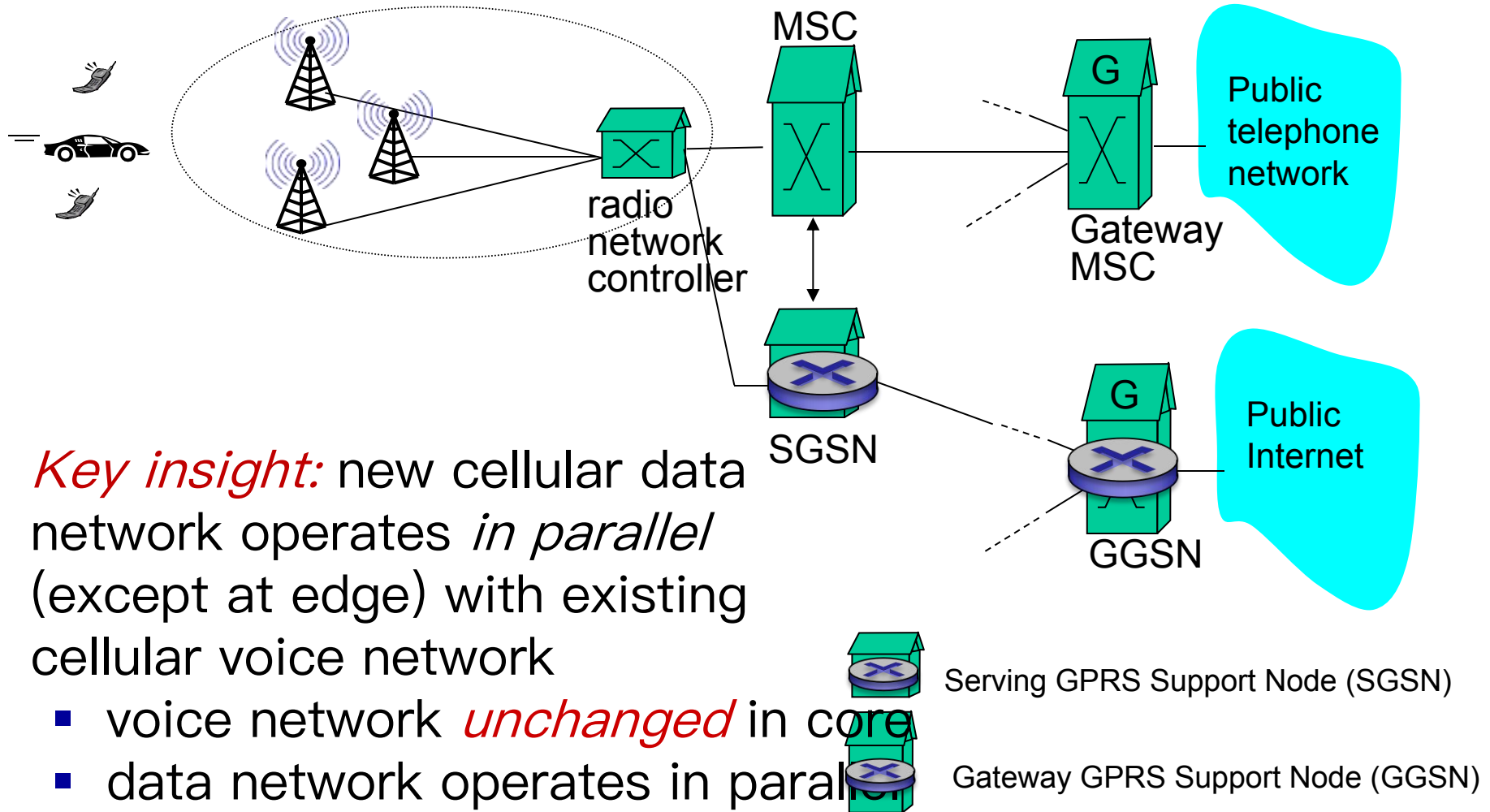
frequency bands



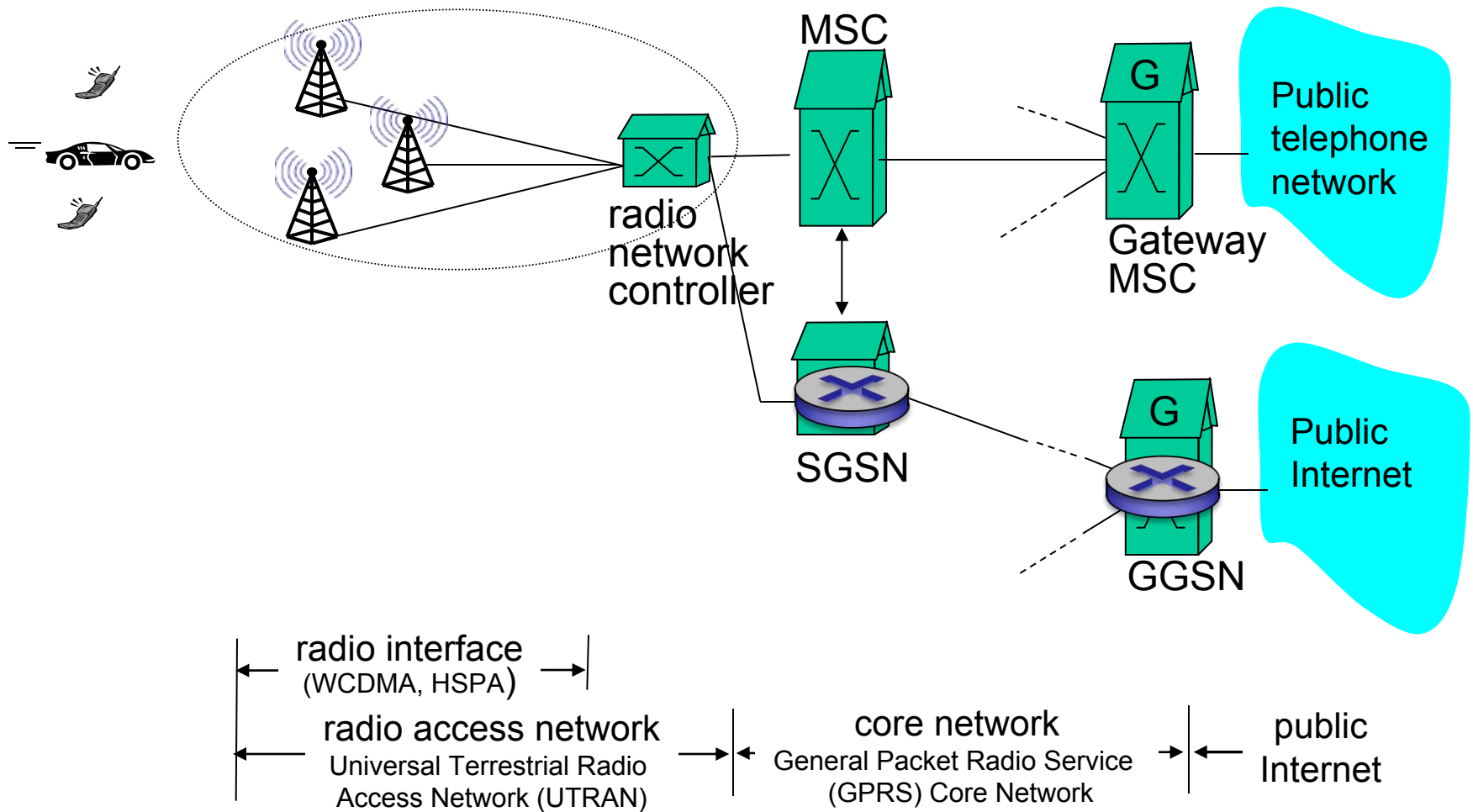
2G (voice) network architecture



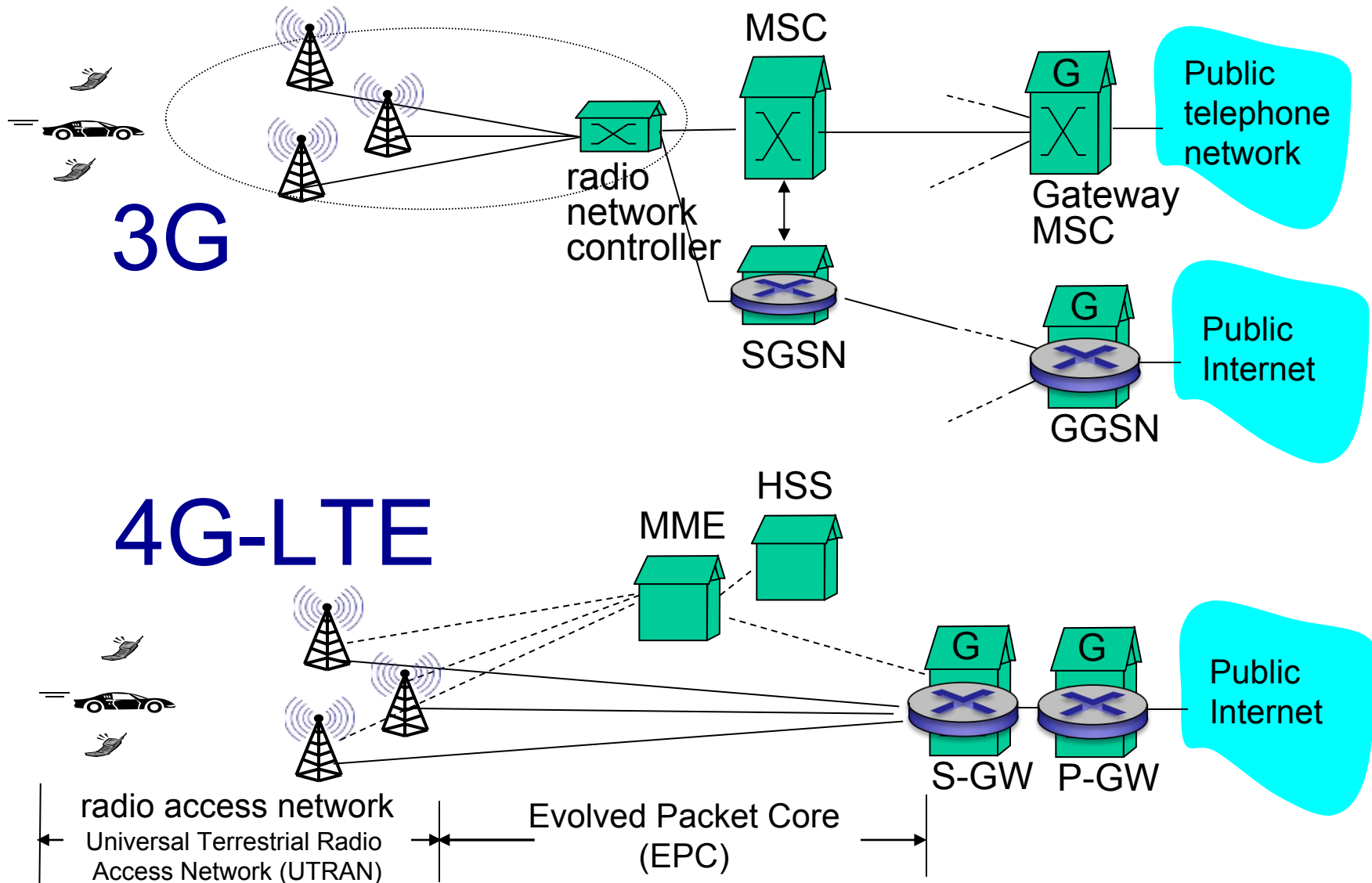
3G (voice+data) network architecture



3G (voice+data) network architecture

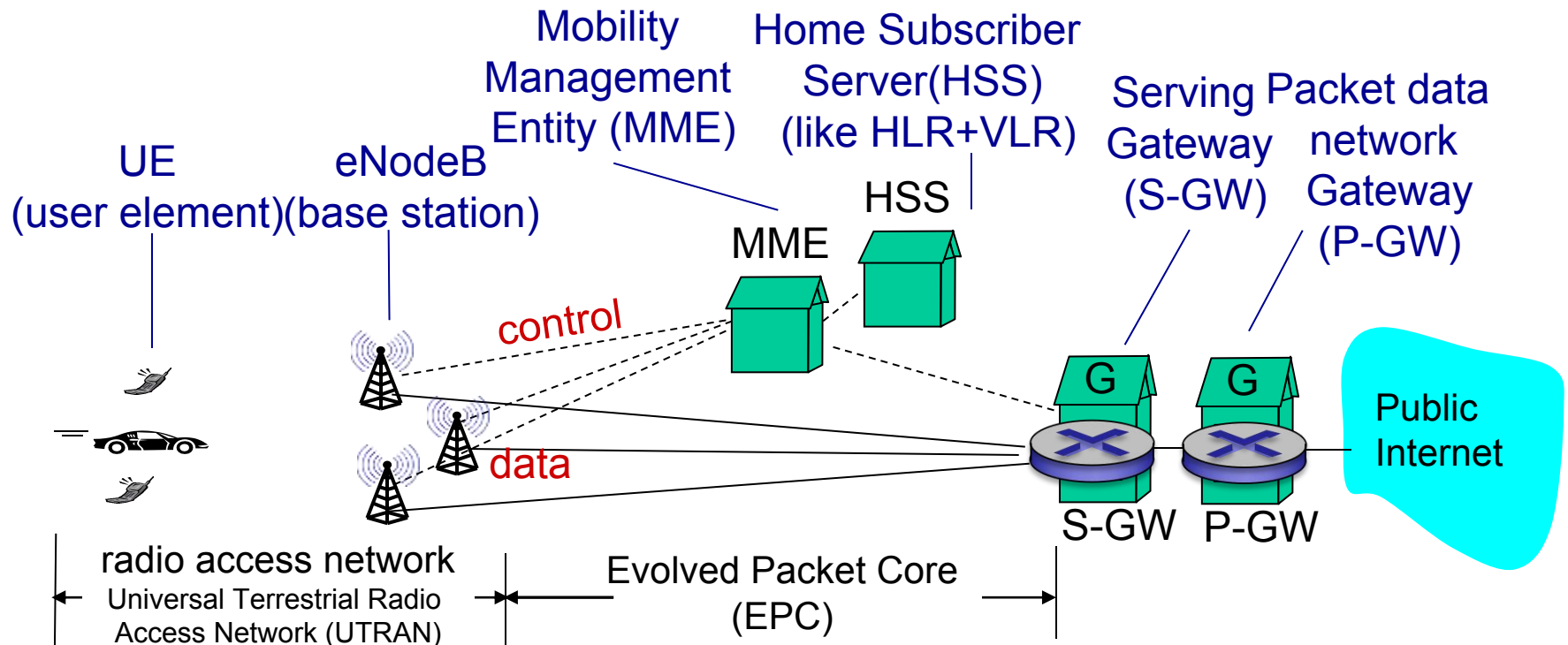


3G versus 4G LTE network architecture



4G: differences from 3G

- all IP core: IP packets tunneled (through core IP network) from base station to gateway
- no separation between voice and data — all traffic carried over IP core to gateway



Chapter 7 outline

7.1 Introduction

Wireless

7.2 Wireless links, characteristics

- CDMA

7.3 IEEE 802.11 wireless LANs (“Wi-Fi”)

7.4 Cellular Internet Access

- architecture
- standards (e.g., 3G, LTE)

Mobility

7.5 Principles: addressing and routing to mobile users

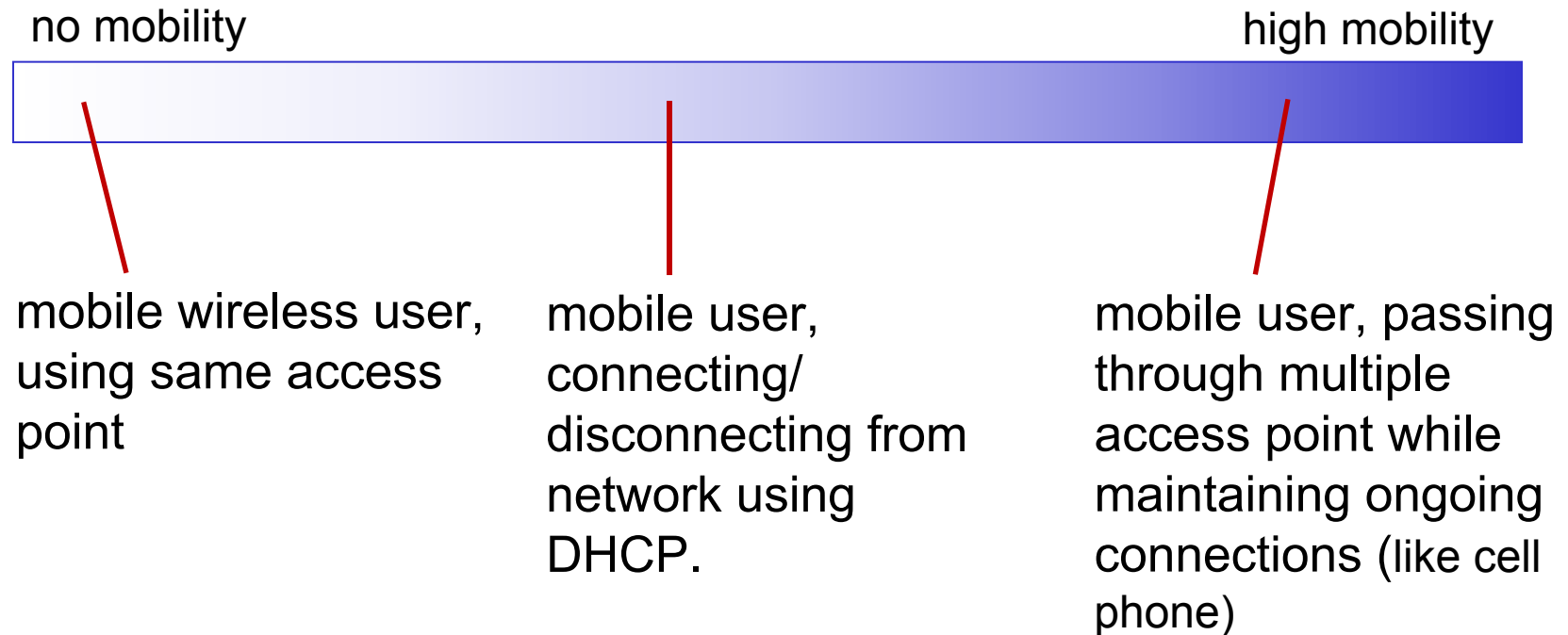
7.6 Mobile IP

7.7 Handling mobility in cellular networks

7.8 Mobility and higher-layer protocols

What is mobility?

- spectrum of mobility, from the *network* perspective:

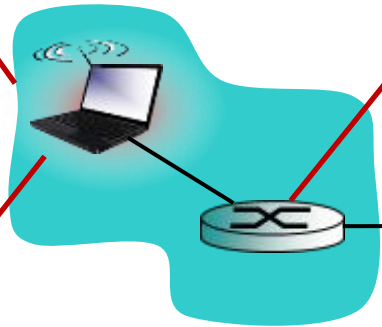


Mobility: vocabulary

home network: permanent
“home” of mobile
(e.g., 128.119.40/24)

home agent: entity that will
perform mobility functions on
behalf of mobile, when mobile is
remote

permanent address:
address in home
network, *can always* be
used to reach mobile
e.g., 128.119.40.186



wide area
network



Mobility: more vocabulary

permanent address: remains constant (e.g., 128.119.40.186)

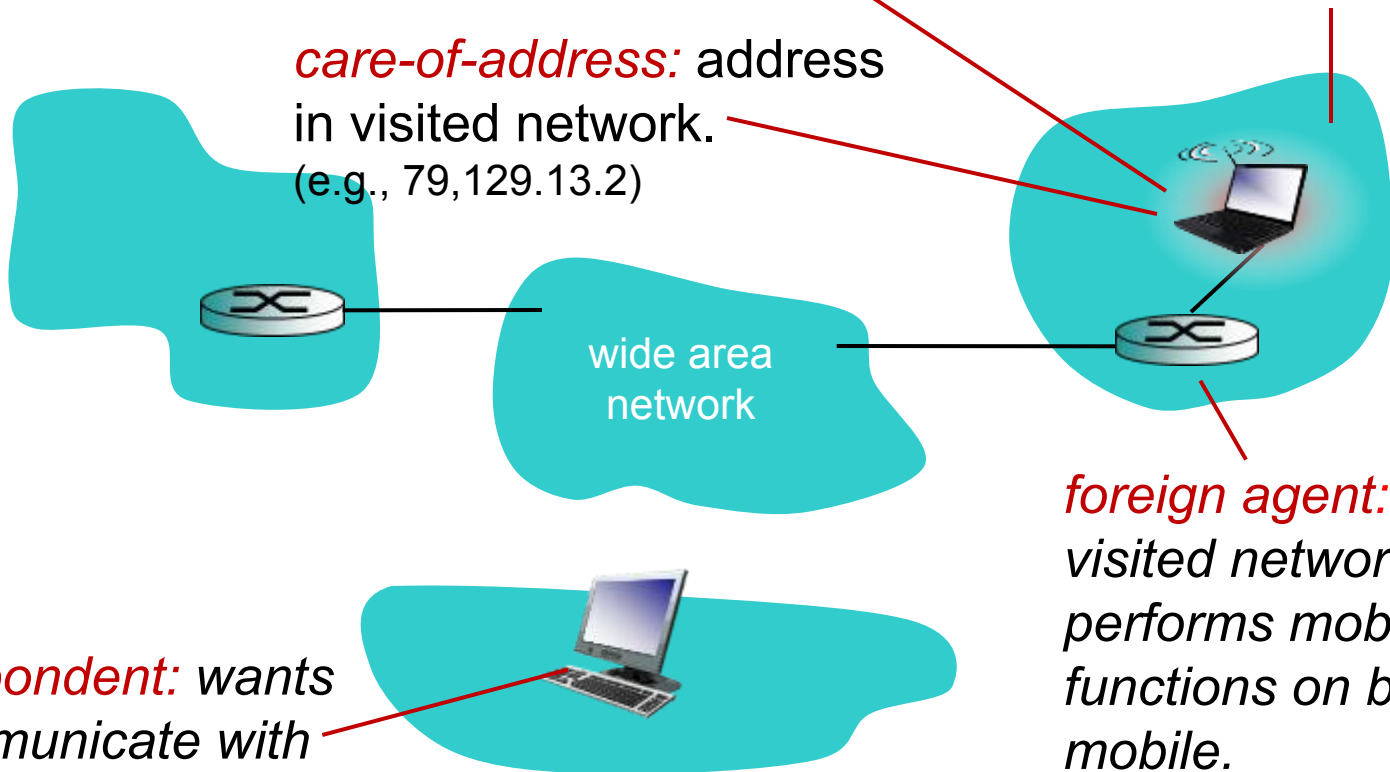
visited network: network in which mobile currently resides (e.g., 79.129.13/24)

care-of-address: address in visited network. (e.g., 79.129.13.2)

wide area network

foreign agent: entity in visited network that performs mobility functions on behalf of mobile.

correspondent: wants to communicate with mobile



How do *you* contact a mobile friend:

Consider friend frequently changing addresses, how do you find her?

- search all phone books?
- call her parents?
- expect her to let you know where he/she is'
- Facebook!



Mobility: approaches

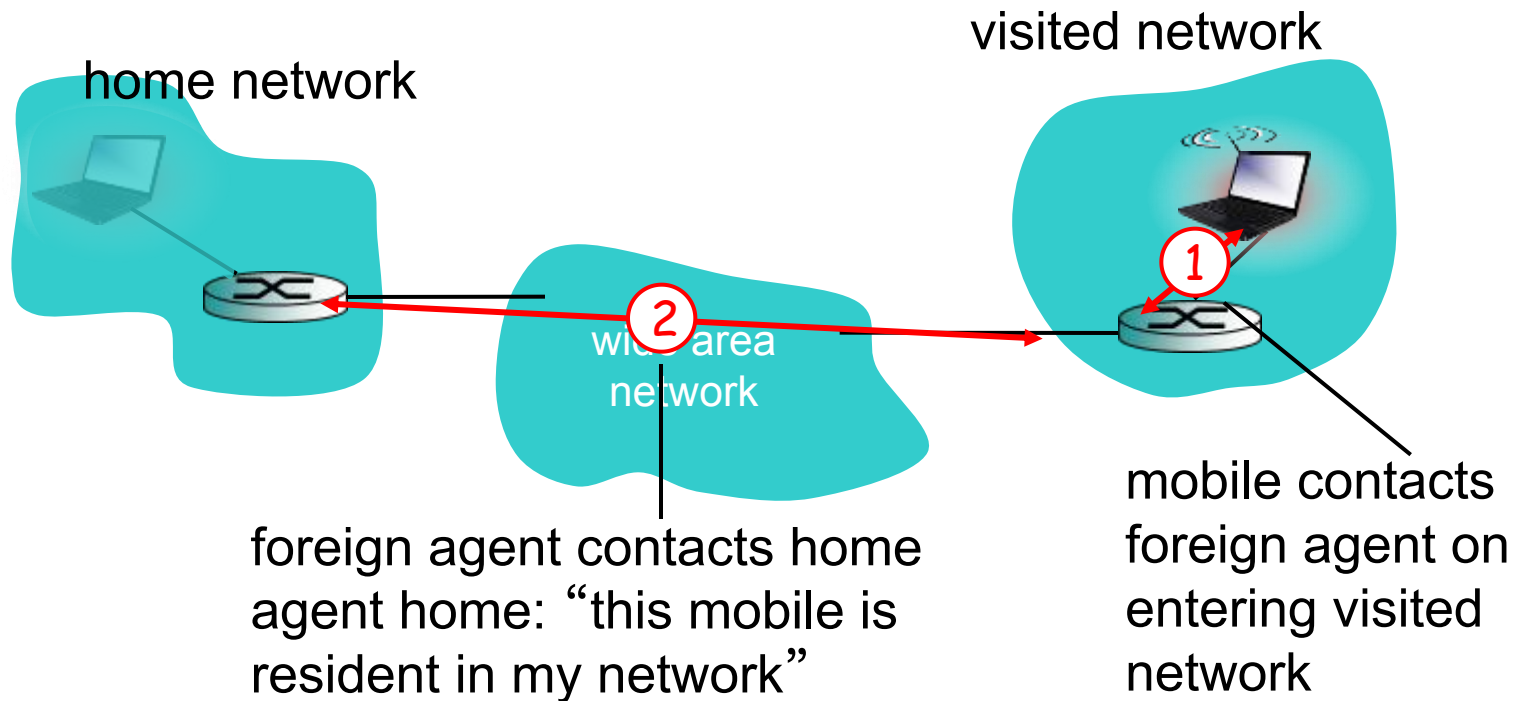
- *let routing handle it:* routers advertise permanent address of mobile-nodes-in-residence via usual routing table exchange.
 - routing tables indicate where each mobile located
 - no changes to end-systems
- *let end-systems handle it:*
 - *indirect routing:* communication from correspondent to mobile goes through home agent, then forwarded to remote
 - *direct routing:* correspondent gets foreign address of mobile, sends directly to mobile

Mobility: approaches

- *let routing handle it:* routers advertise permanent address of mobile-nodes-in-residence via routing table exchange.
 - routing table exchange where each mobile located
 - no changes to end-systems
- *let end-systems handle it:*
 - *indirect routing:* communication from correspondent to mobile goes through home agent, then forwarded to remote
 - *direct routing:* correspondent gets foreign address of mobile, sends directly to mobile

not
scalable
to millions of
mobiles

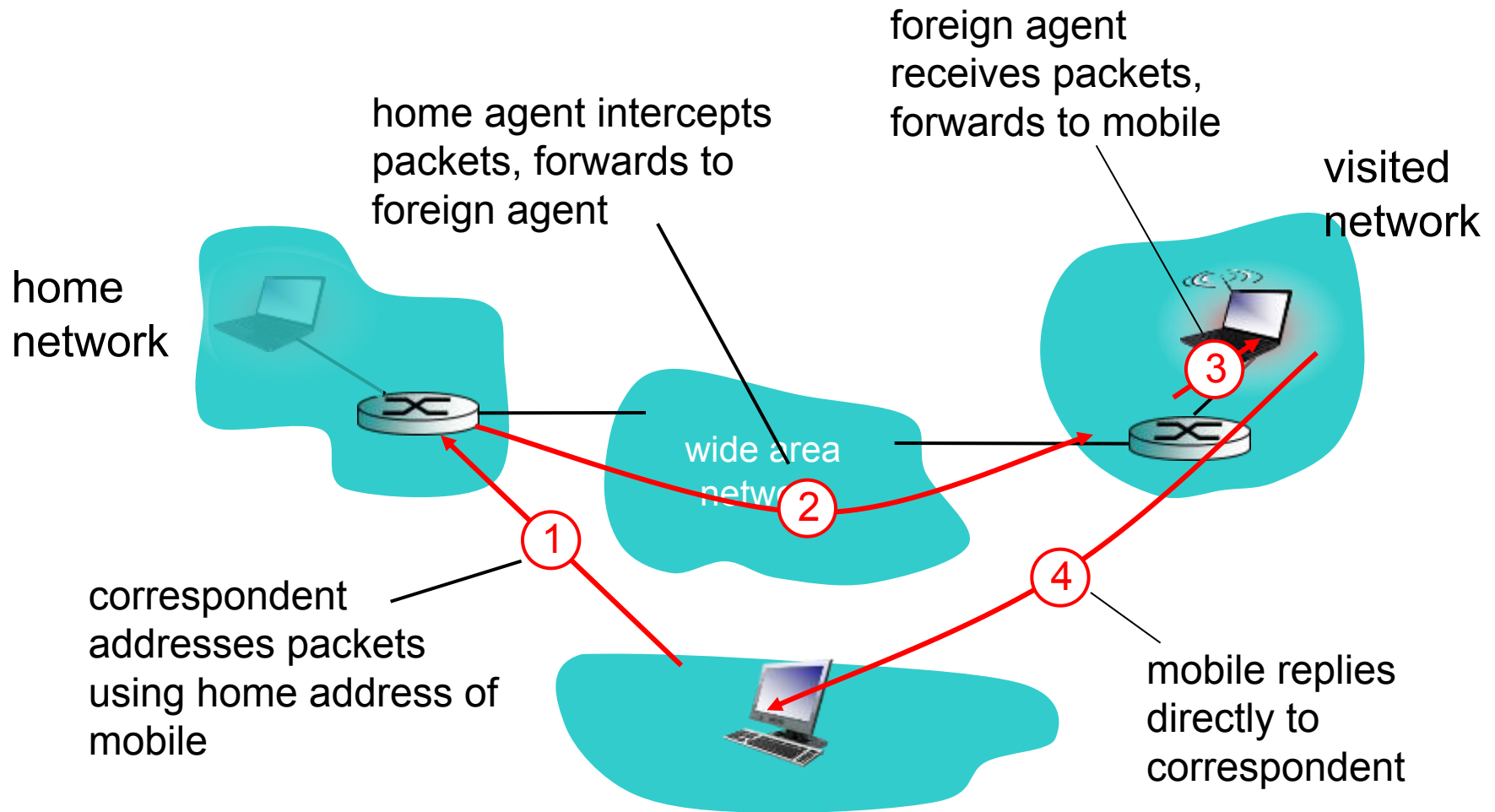
Mobility: registration



end result:

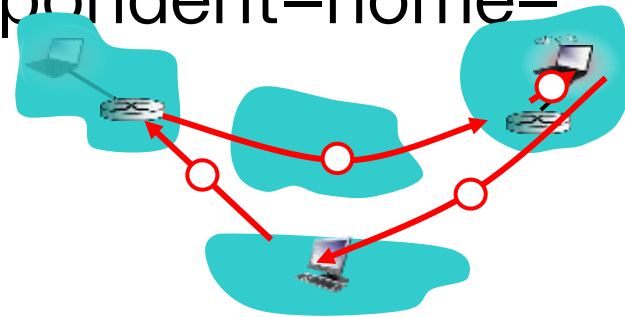
- foreign agent knows about mobile
- home agent knows location of mobile

Mobility via indirect routing



Indirect Routing: comments

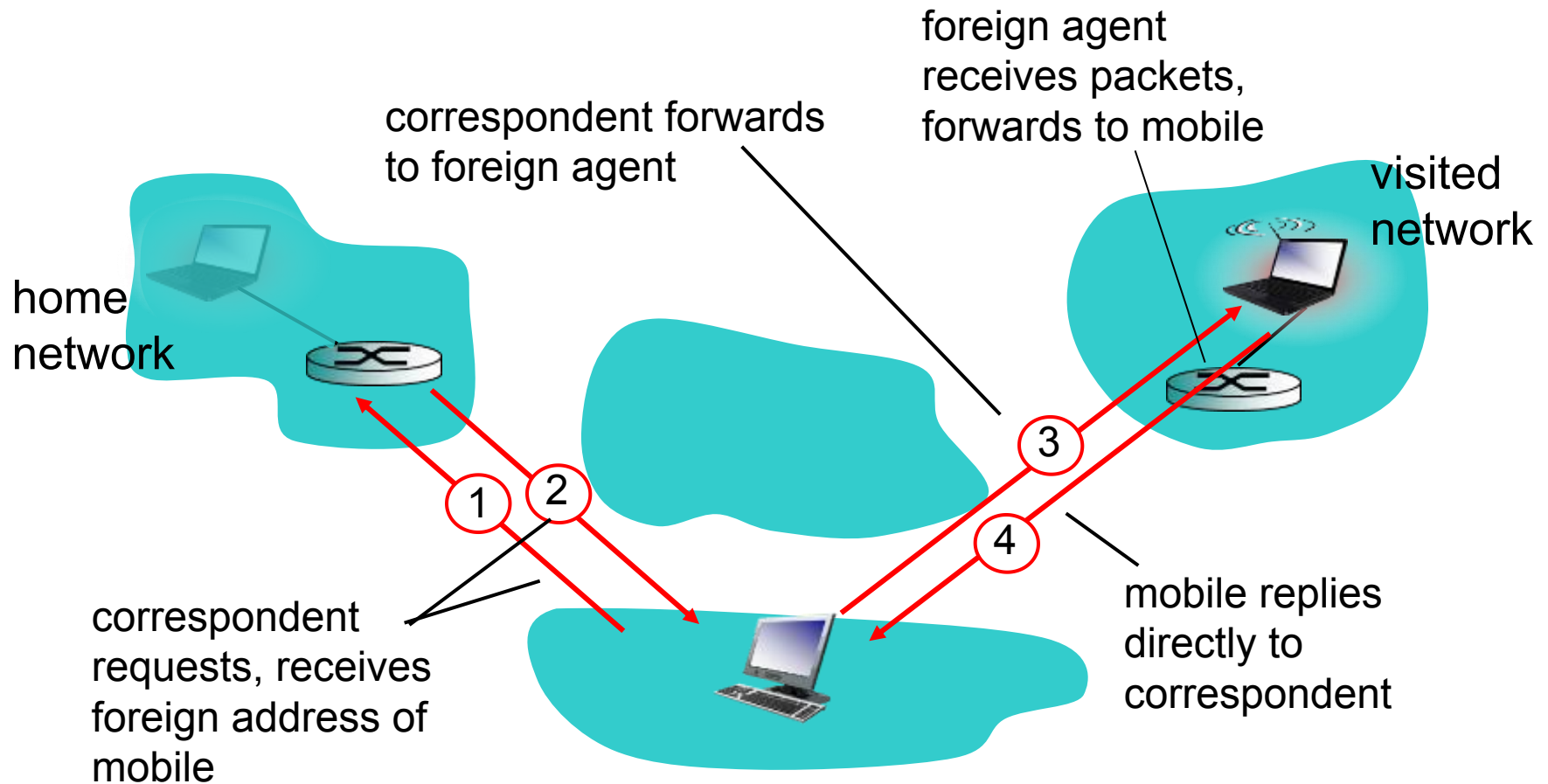
- mobile uses two addresses:
 - **permanent address:** used by correspondent (hence mobile location is *transparent* to correspondent)
 - **care-of-address:** used by home agent to forward datagrams to mobile
- foreign agent functions may be done by mobile itself
- **triangle routing:** correspondent-home-network-mobile
 - inefficient when correspondent, mobile are in same network



Indirect routing: moving between networks

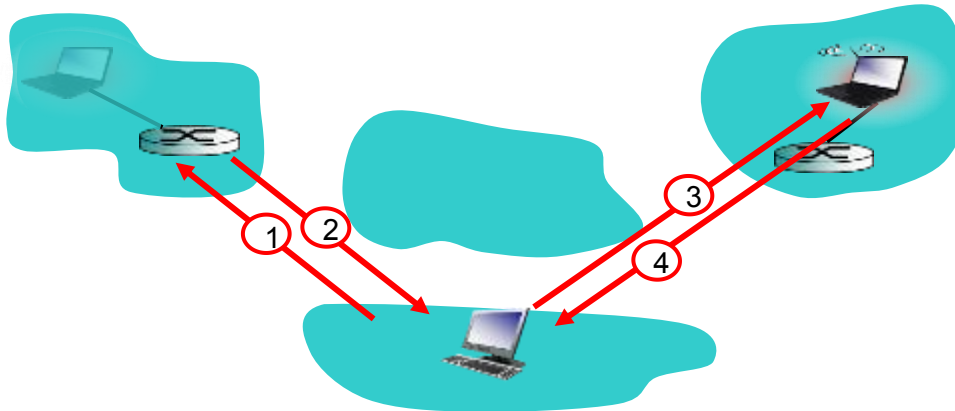
- suppose mobile user moves to another network
 - registers with new foreign agent
 - new foreign agent registers with home agent
 - home agent update care-of-address for mobile
 - packets continue to be forwarded to mobile (but with new care-of-address)
- mobility, changing foreign networks transparent: *on going connections can be maintained!*

Mobility via direct routing



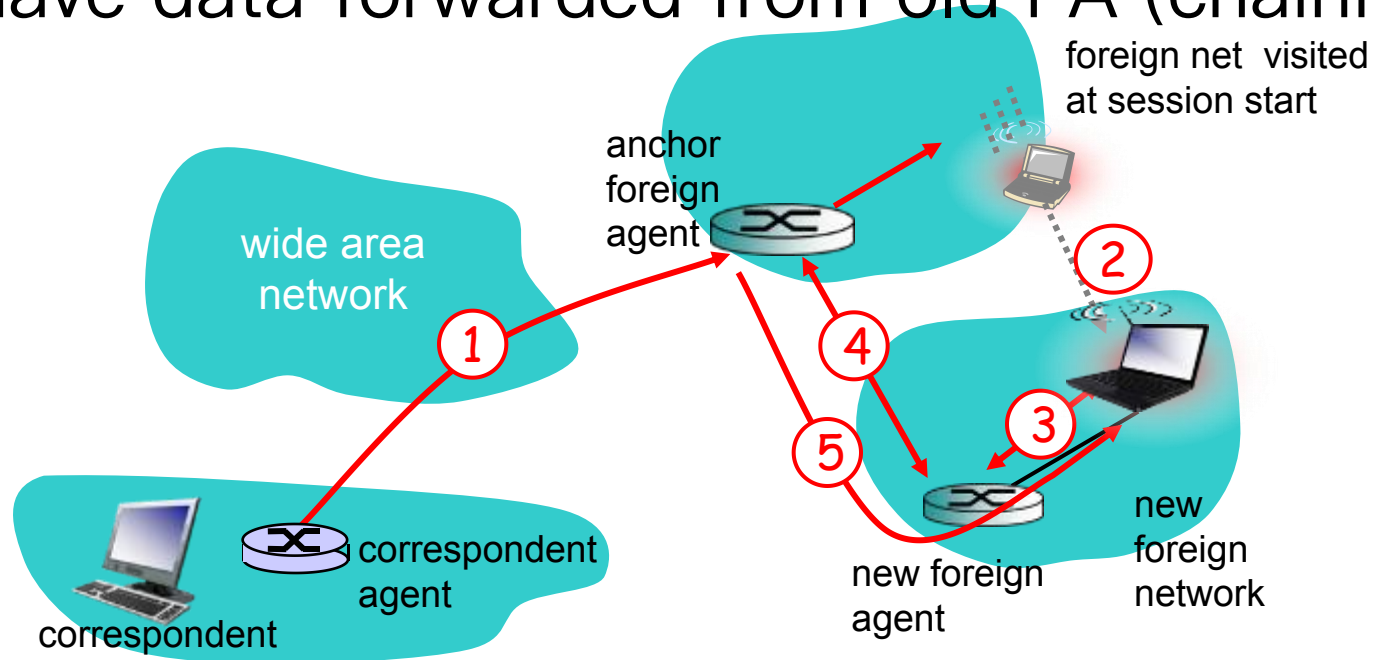
Mobility via direct routing: comments

- overcome triangle routing problem
- *non-transparent to correspondent:*
correspondent must get care-of-address from home agent
 - what if mobile changes visited network?



Accommodating mobility with direct routing

- anchor foreign agent: FA in first visited network
- data always routed first to anchor FA
- when mobile moves: new FA arranges to have data forwarded from old FA (chaining)



Chapter 7 summary

Wireless

- wireless links:
 - capacity, distance
 - channel impairments
 - CDMA
- IEEE 802.11 (“Wi-Fi”)
 - CSMA/CA reflects wireless channel characteristics
- cellular access
 - architecture
 - standards (e.g., 3G, 4G LTE)

Mobility

- principles: addressing, routing to mobile users
 - home, visited networks
 - direct, indirect routing
 - care-of-addresses

Chapter 8 Security

Chapter goals:

- understand principles of network security:
 - cryptography and its *many* uses beyond “confidentiality”
 - authentication
 - message integrity
- security in practice:
 - firewalls and intrusion detection systems
 - security in application, transport, network, link layers

Chapter 8 outline

- What is network security?
- Principles of cryptography
- Message integrity, authentication
- Securing e-mail
- Securing TCP connections: TLS
- Network layer security: IPsec
- Security in wireless and mobile networks
- Operational security: firewalls and IDS



What is network security?

confidentiality: only sender, intended receiver should “understand” message contents

- sender encrypts message
- receiver decrypts message

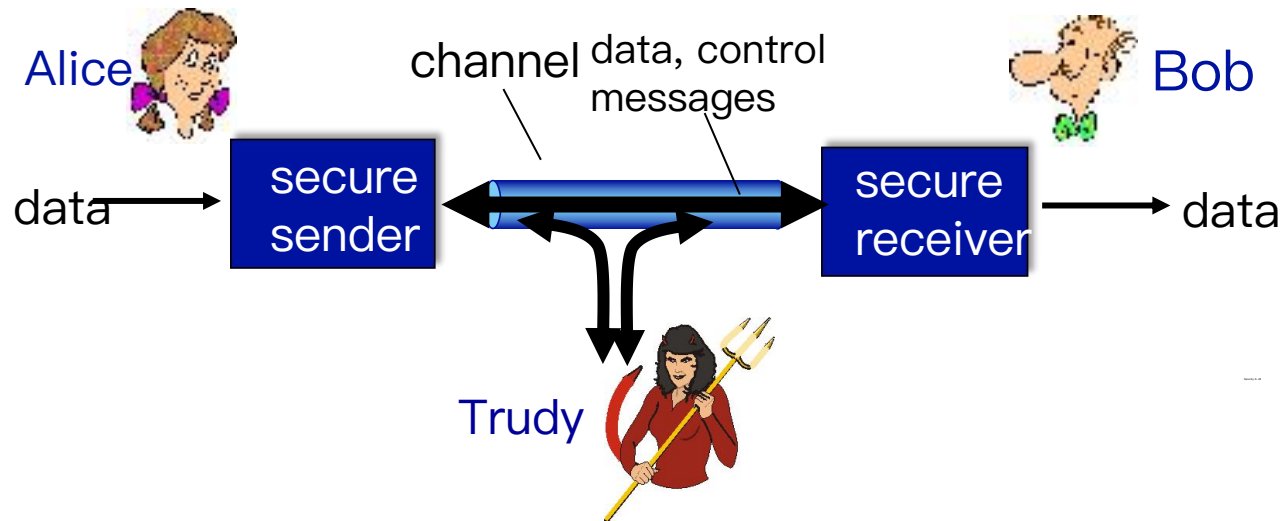
authentication: sender, receiver want to confirm identity of each other

message integrity: sender, receiver want to ensure message not altered (in transit, or afterwards) without detection

access and availability: services must be accessible and available to users

Friends and enemies: Alice, Bob, Trudy

- well-known in network security world
- Bob, Alice (lovers!) want to communicate “securely”
- Trudy (intruder) may intercept, delete, add messages



Friends and enemies: Alice, Bob, Trudy

Who might Bob and Alice be?

- ... well, *real-life* Bobs and Alices!
- Web browser/server for electronic transactions (e.g., on-line purchases)
- on-line banking client/server
- DNS servers
- BGP routers exchanging routing table updates
- other examples?

There are bad guys (and girls) out there!

Q: What can a “bad guy” do?

A: A lot! (recall section 1.6)

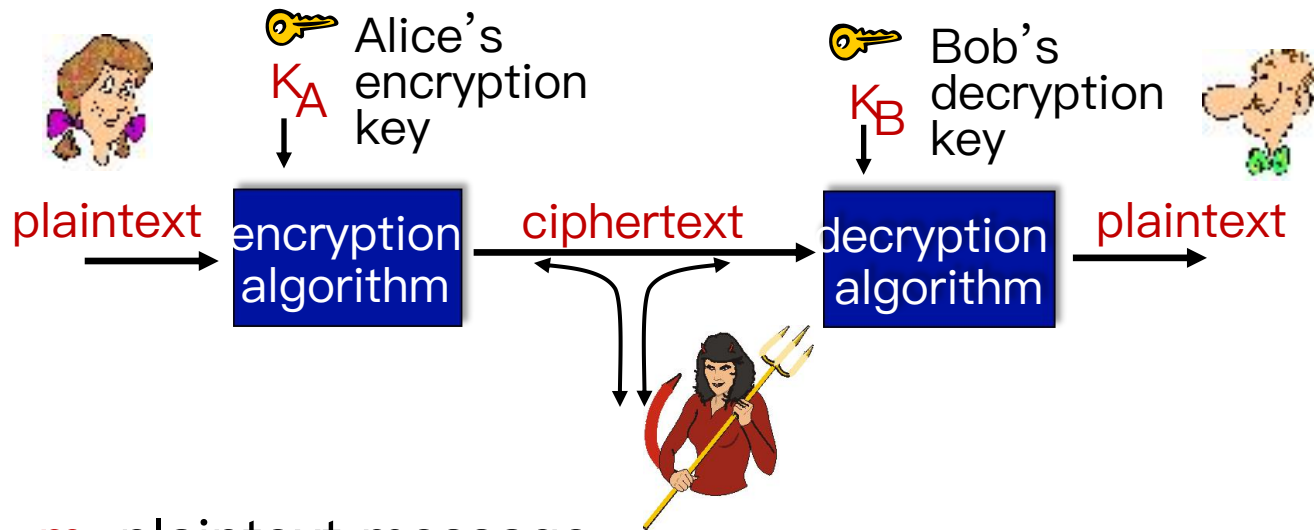
- **eavesdrop:** intercept messages
- actively **insert** messages into connection
- **impersonation:** can fake (spoof) source address in packet (or any field in packet)
- **hijacking:** “take over” ongoing connection by removing sender or receiver, inserting himself in place
- **denial of service:** prevent service from being used by others (e.g., by overloading resources)

Chapter 8 outline

- What is network security?
- Principles of cryptography
- Message integrity, authentication
- Securing e-mail
- Securing TCP connections: TLS
- Network layer security: IPsec
- Security in wireless and mobile networks
- Operational security: firewalls and IDS



The language of cryptography



m : plaintext message

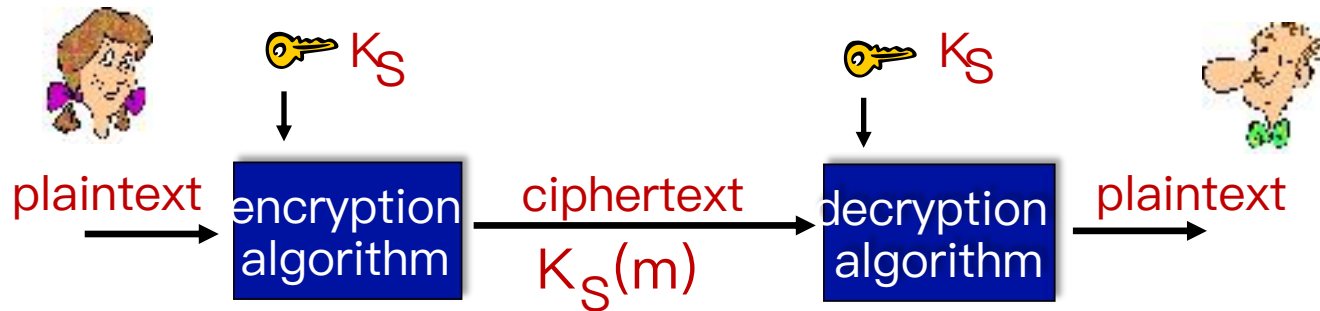
$K_A(m)$: ciphertext, encrypted with key K_A

$m = K_B(K_A(m))$

Breaking an encryption scheme

- **cipher-text only attack:** Trudy has ciphertext she can analyze
- **two approaches:**
 - brute force: search through all keys
 - statistical analysis
- **known-plaintext attack:** Trudy has plaintext corresponding to ciphertext
 - *e.g.*, in monoalphabetic cipher, Trudy determines pairings for a,l,i,c,e,b,o,
- **chosen-plaintext attack:** Trudy can get ciphertext for chosen plaintext

Symmetric key cryptography



symmetric key crypto: Bob and Alice share same (symmetric) key: K

- e.g., key is knowing substitution pattern in mono alphabetic substitution cipher

Q: how do Bob and Alice agree on key value?

Simple encryption scheme

substitution cipher: substituting one thing for another

- monoalphabetic cipher: substitute one letter for another

plaintext:	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
		↓																								↓
ciphertext:	m	n	b	v	c	x	z	a	s	d	f	g	h	j	k	l	p	o	i	u	y	t	r	e	w	q

e.g.: Plaintext: bob. i love you. alice
ciphertext: nkn. s gktc wky. mgsbc

🔑 *Encryption key*: mapping from set of 26 letters
to set of 26 letters

A more sophisticated encryption

- n substitution ciphers, $M_1, M_2, \dots, M_n$
- cycling pattern:
 - e.g., $n=4$: M_1, M_3, M_4, M_3, M_2 ; M_1, M_3, M_4, M_3, M_2 ; ..
- for each new plaintext symbol, use subsequent substitution pattern in cyclic pattern
 - dog: d from M_1 , o from M_3 , g from M_4



Encryption key: n substitution ciphers, and cyclic pattern

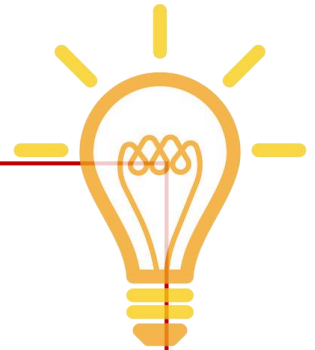
Public Key Cryptography

symmetric key crypto:

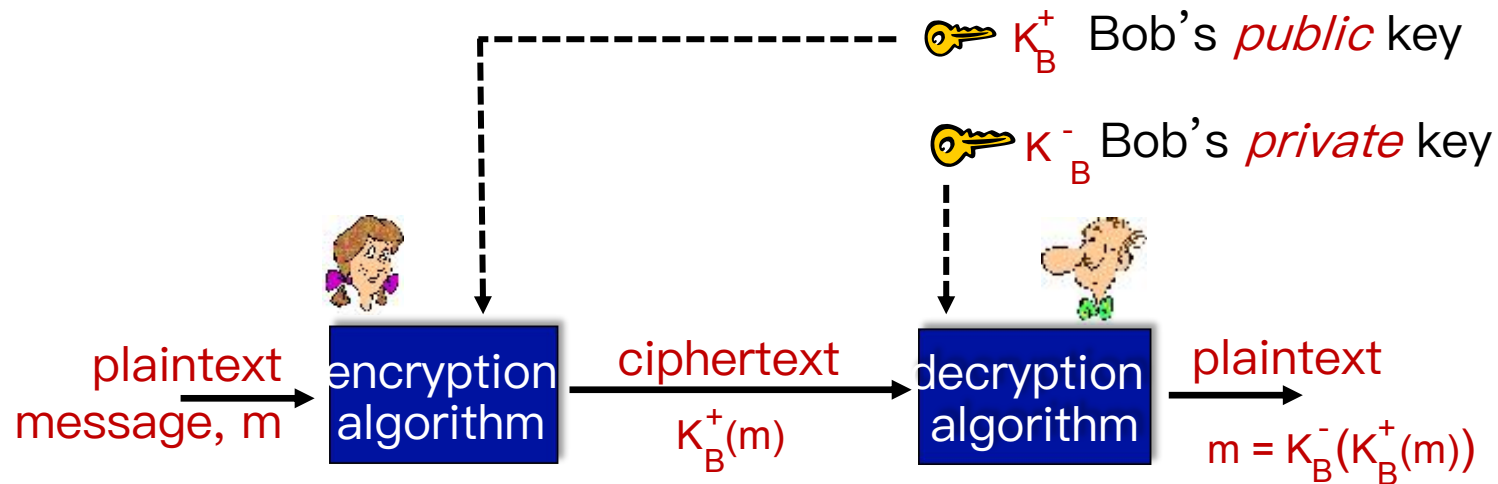
- requires sender, receiver know shared secret key
- Q: how to agree on key in first place (particularly if never “met”)?

public key crypto

- *radically* different approach [Diffie-Hellman76, RSA78]
- sender, receiver do *not* share secret key
- *public* encryption key known to *all*
- *private* decryption key known only to receiver



Public Key Cryptography



Wow - public key cryptography revolutionized 2000-year-old (previously only symmetric key) cryptography!

- similar ideas emerged at roughly same time, independently in US and UK (classified)

Public key encryption algorithms

requirements:

- ① need $K_B^+(\cdot)$ and $K_B^-(\cdot)$ such that

$$K_B^-(K_B^+(m)) = m$$

- ② given public key K_B^+ , it should be impossible to compute private key K_B^-

RSA: Rivest, Shamir, Adelson algorithm

RSA: Creating public/private key pair

1. choose two large prime numbers p, q . (e.g., 1024 bits each)
2. compute $n = pq$, $z = (p-1)(q-1)$
3. choose e (with $e < n$) that has no common factors with z (e, z are “relatively prime”).
4. choose d such that $ed-1$ is exactly divisible by z . (in other words: $ed \bmod z = 1$).
5. *public* key is $\underbrace{(n, e)}_{K_B^+}$. *private* key is $\underbrace{(n, d)}_{K_B^-}$.

Just for your interests. It will not be covered in exam.

RSA: encryption, decryption

0. given (n, e) and (n, d) as computed above
1. to encrypt message $m (< n)$, compute
$$c = m^e \bmod n$$
2. to decrypt received bit pattern, c , compute
$$m = c^d \bmod n$$

magic happens! $m = \underbrace{(m^e \bmod n)}_c \bmod^d n$

Just for your interests. It will not be covered in exam.

RSA: another important property

The following property will be *very* useful later:

$$K_B^-(K_B^+(m)) = m = K_B^+(K_B^-(m))$$

use public
key first,
followed by
private key

use private
key first,
followed by
public key

result is the same!

Why is RSA secure?

- suppose you know Bob's public key (n,e) . How hard is it to determine d ?
- essentially need to find factors of n without knowing the two factors p and q
 - fact: factoring a big number is hard

RSA in practice: session keys

- exponentiation in RSA is computationally intensive
- DES is at least 100 times faster than RSA
- use public key crypto to establish secure connection, then establish second key — symmetric session key — for encrypting data

session key, K_S

- Bob and Alice use RSA to exchange a symmetric session key K_S
- once both have K_S , they use symmetric key cryptography

Chapter 8 outline

- What is network security?
- Principles of cryptography
- **Authentication, message integrity**
- Securing e-mail
- Securing TCP connections: TLS
- Network layer security: IPsec
- Security in wireless and mobile networks
- Operational security: firewalls and IDS



Authentication

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap1.0: Alice says “I am Alice”



failure scenario??



Authentication

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap1.0: Alice says “I am Alice”



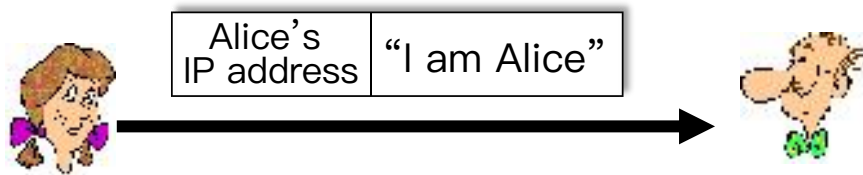
*in a network,
Bob can not
“see” Alice,
so Trudy
simply
declares
herself to be
Alice*



Authentication: another try

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap2.0: Alice says “I am Alice” in an IP packet containing her source IP address



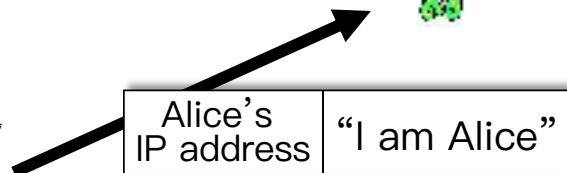
failure scenario??



Authentication: another try

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap2.0: Alice says “I am Alice” in an IP packet containing her source IP address

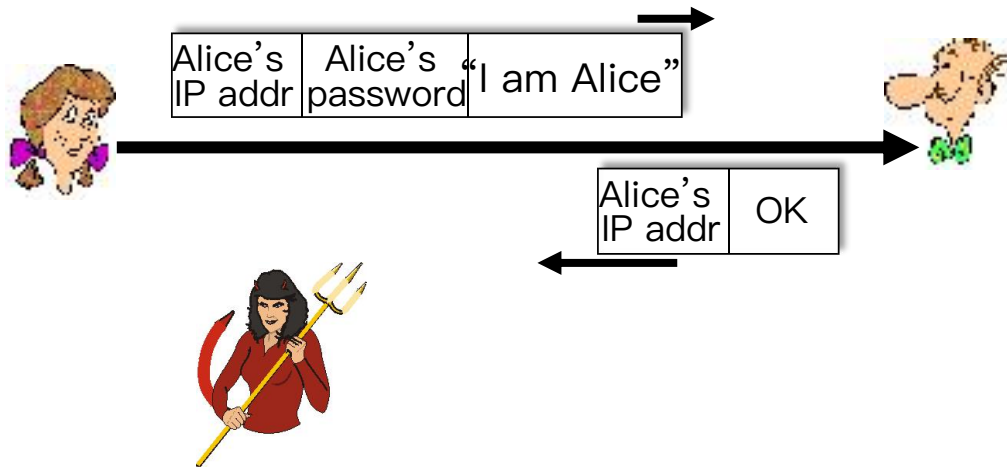


*Trudy can create
a packet “spoofing”
Alice’s address*

Authentication: a third try

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap3.0: Alice says “I am Alice” and sends her secret password to “prove” it.

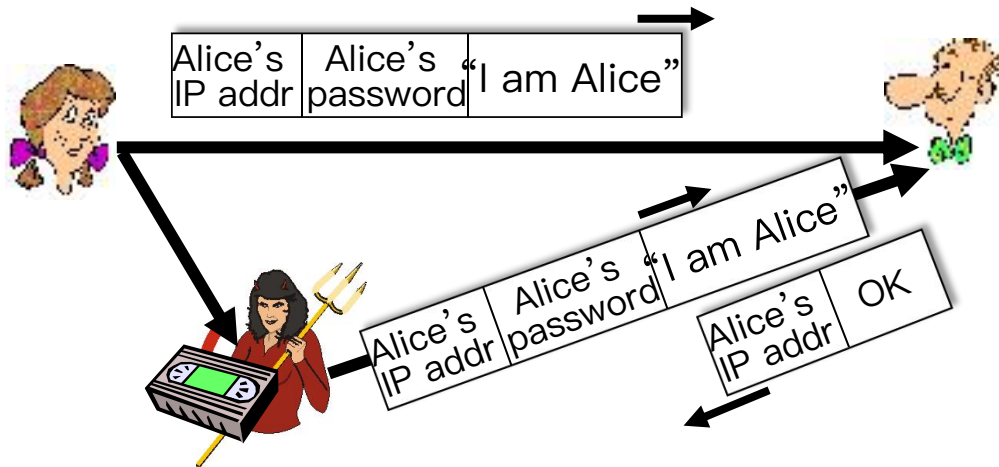


failure scenario??

Authentication: a third try

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap3.0: Alice says “I am Alice” and sends her secret password to “prove” it.

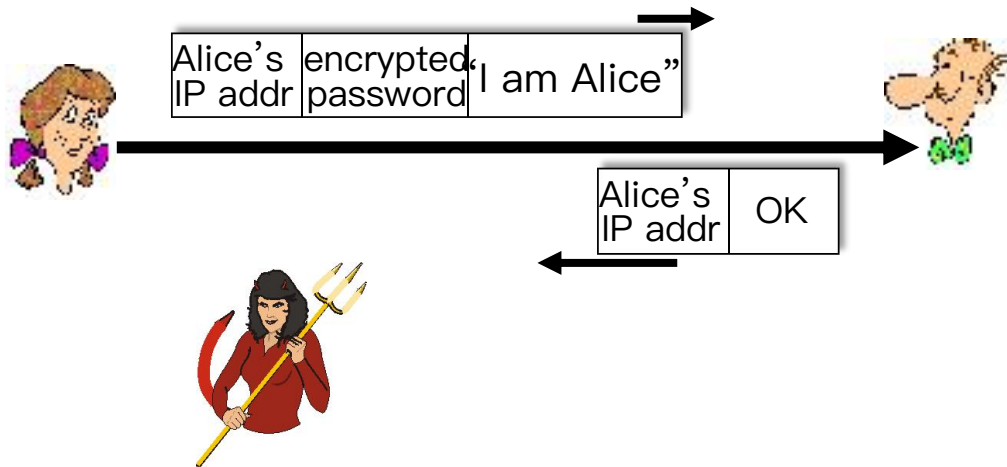


*playback
attack: Trudy
records Alice's
packet
and later
plays it back to
Bob*

Authentication: a modified third try

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap3.0: Alice says “I am Alice” and sends her encrypted secret password to “prove” it.

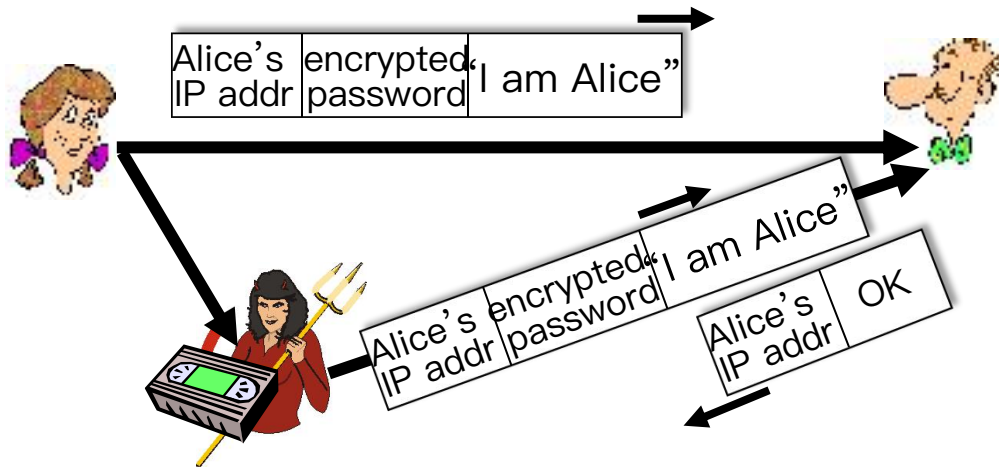


failure scenario??

Authentication: a modified third try

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap3.0: Alice says “I am Alice” and sends her encrypted secret password to “prove” it.



*playback attack
still works: Trudy
records Alice's
packet
and later plays it
back to Bob*

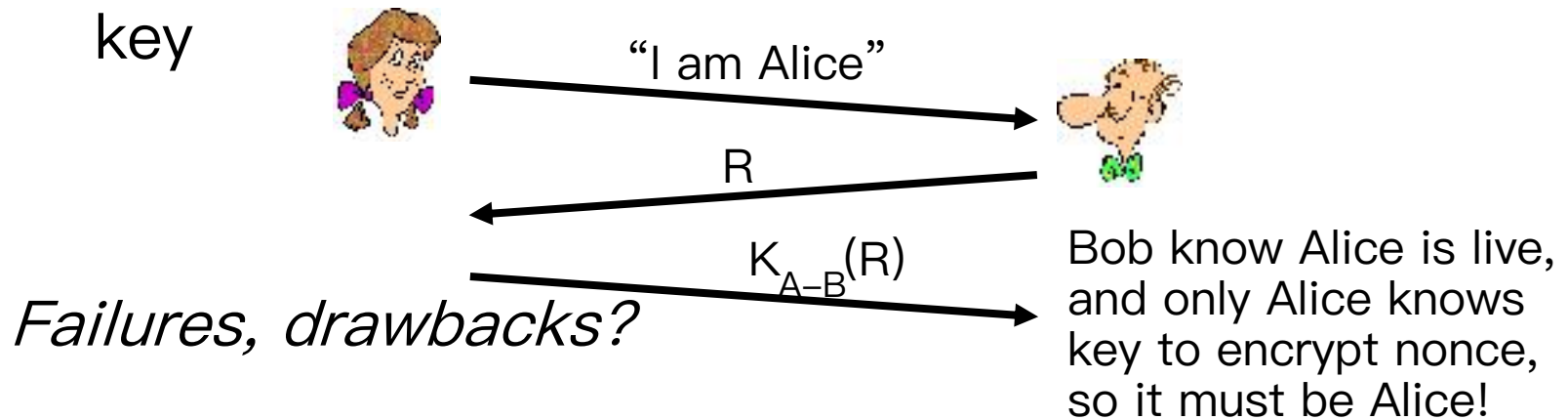
Authentication: a fourth try

Goal: avoid playback attack

nonce: number (R) used only **once-in-a-lifetime**

protocol ap4.0: to prove Alice “live”, Bob sends Alice nonce, R

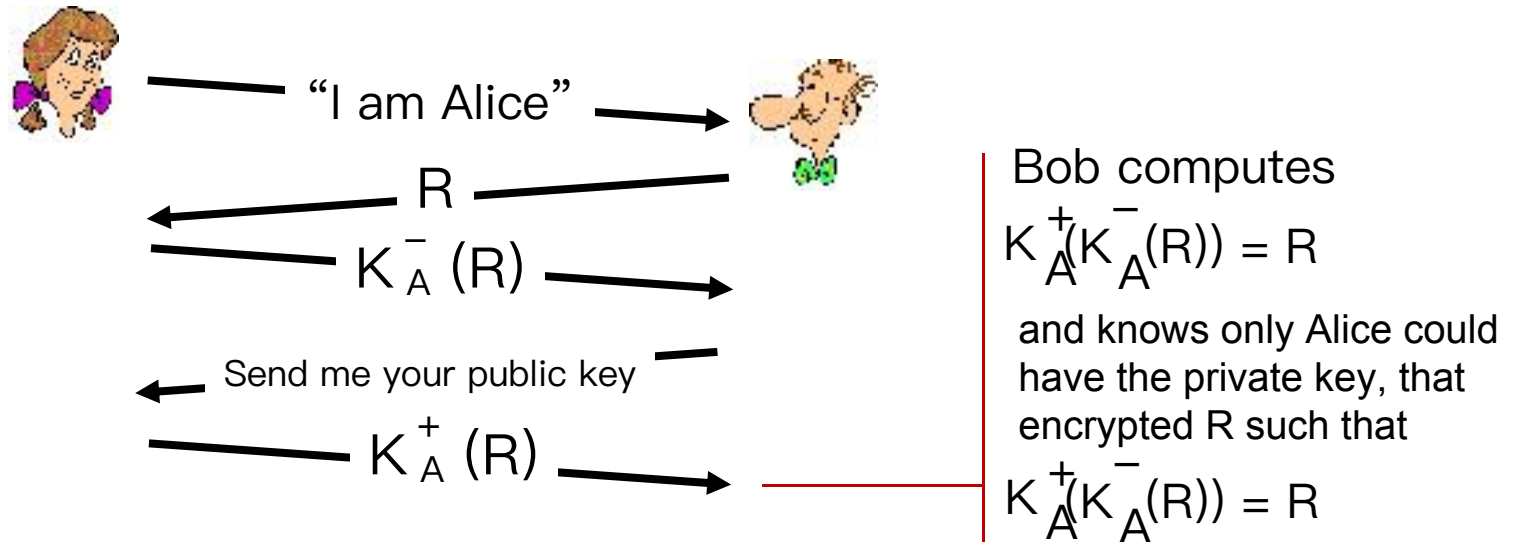
- Alice must return R, encrypted with shared secret key



Authentication: ap5.0

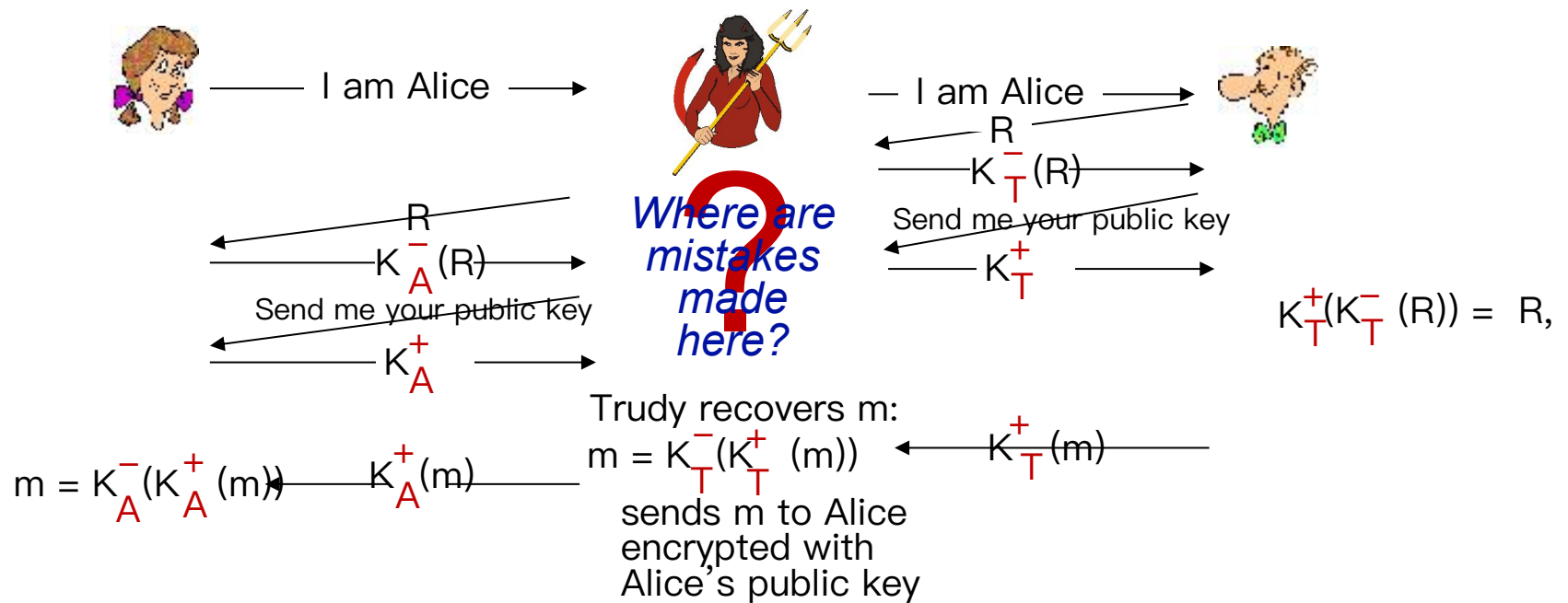
ap4.0 requires shared symmetric key – can we authenticate using public key techniques?

ap5.0: use nonce, public key cryptography



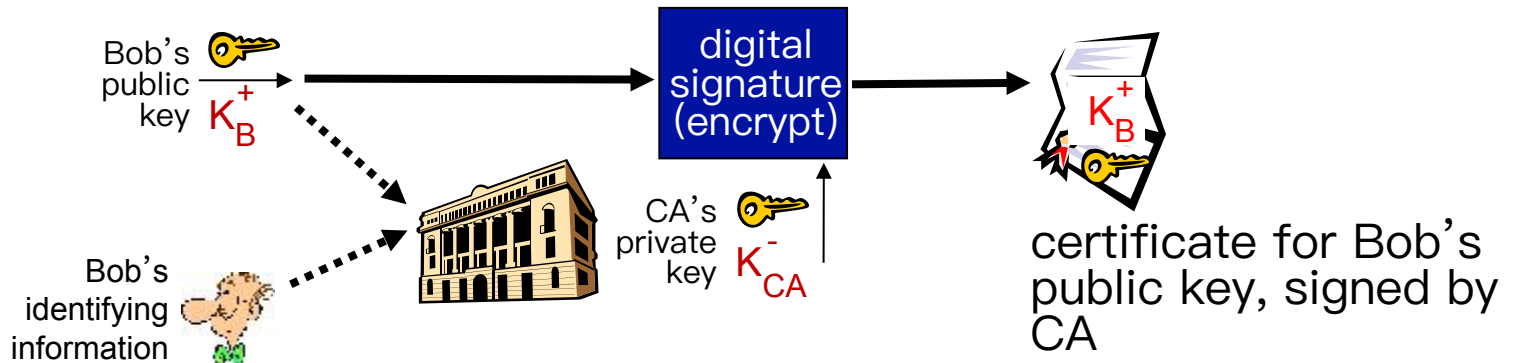
Authentication: ap5.0 — there's still a flaw!

man (or woman) in the middle attack: Trudy poses as Alice (to Bob) and as Bob (to Alice)



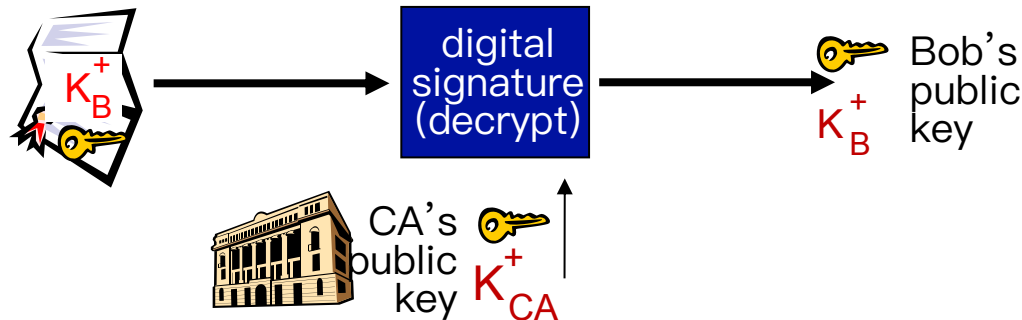
Public key Certification Authorities (CA)

- **certification authority (CA):** binds public key to particular entity, E
- entity (person, website, router) registers its public key with CA provides “proof of identity” to CA
 - CA creates certificate binding identity E to E’s public key
 - certificate containing E’s public key digitally signed by CA: CA says “this is E’s public key”



Public key Certification Authorities (CA)

- when Alice wants Bob's public key:
 - gets Bob's certificate (Bob or elsewhere)
 - apply CA's public key to Bob's certificate, get Bob's public key



Chapter 8 outline

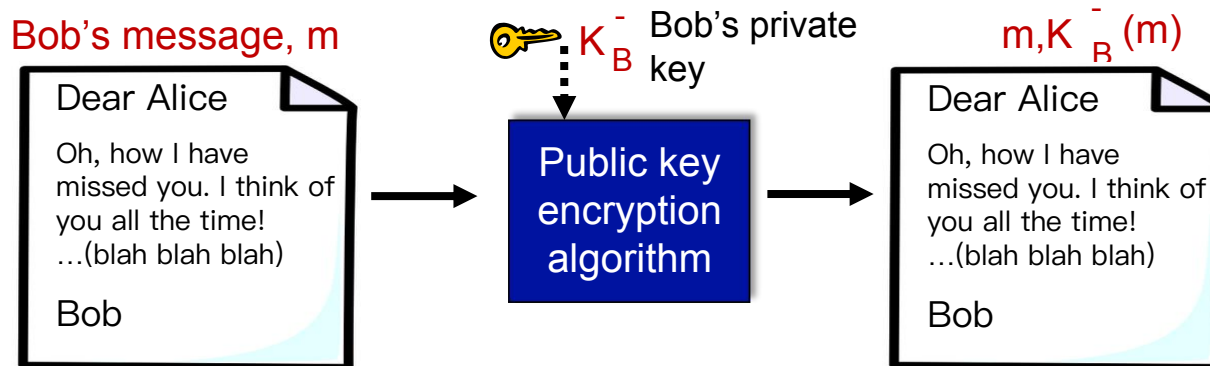
- What is network security?
- Principles of cryptography
- Authentication, **message integrity**
- Securing e-mail
- Securing TCP connections: TLS
- Network layer security: IPsec
- Security in wireless and mobile networks
- Operational security: firewalls and IDS



Digital signatures

cryptographic technique analogous to hand-written signatures:

- sender (Bob) digitally signs document: he is document owner/creator.
- *verifiable, nonforgeable*: recipient (Alice) can prove to someone that Bob, and no one else (including Alice), must have signed document
- **simple digital signature for message m :**
 - Bob signs m by encrypting with his private key K_B , creating “signed” message, $K_B^-(m)$



Digital signatures

- suppose Alice receives msg m , with signature: $m, K_B(m)$
- Alice verifies m signed by Bob by applying Bob's public key K_B^+ to $K_B^-(m)$ then checks $K_B^+(K_B^-(m)) = m$.
- If $K_B^+(K_B^-(m)) = m$, whoever signed m must have used Bob's private key

Alice thus verifies that:

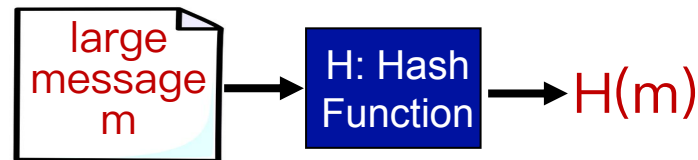
- Bob signed m
- no one else signed m
- Bob signed m and not m'

Message digests

computationally expensive to public-key-encrypt long messages

goal: fixed-length, easy-to-compute digital “fingerprint”

- apply hash function H to m , get fixed size message digest, $H(m)$



Hash function properties:

- many-to-1, but computationally different
- produces fixed-size msg digest (fingerprint)
- given message digest x , computationally infeasible to find m such that $x = H(m)$

Internet checksum: poor crypto hash function

Internet checksum has some properties of hash function:

- produces fixed length digest (16-bit sum) of message
- is many-to-one

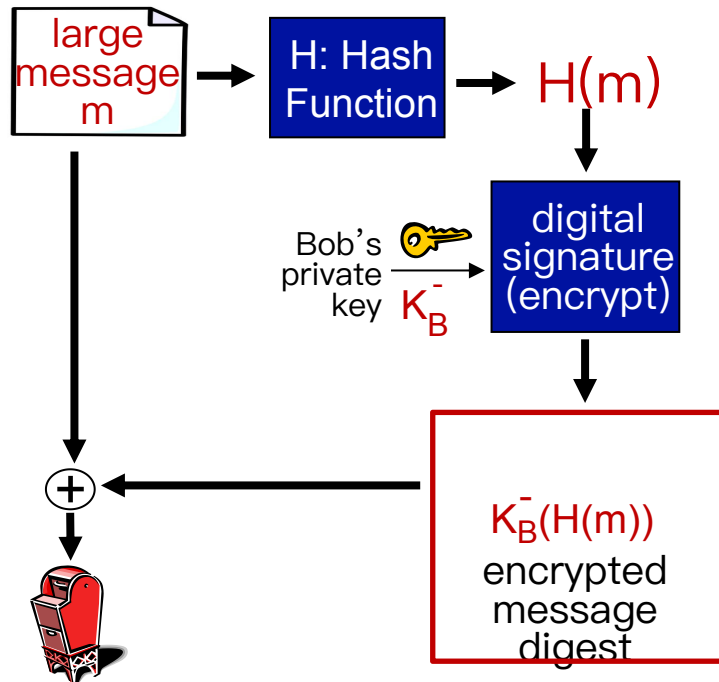
but given message with given hash value, it is easy to find another message with same hash value:

<u>message</u>	<u>ASCII format</u>	<u>message</u>	<u>ASCII format</u>
I O U 1	49 4F 55 31	I O U 9	49 4F 55 39
0 0 . 9	30 30 2E 39	0 0 . 1	30 30 2E 31
9 B O B	39 42 D2 42	9 B O B	39 42 D2 42
B2 C1 D2 AC		B2 C1 D2 AC	

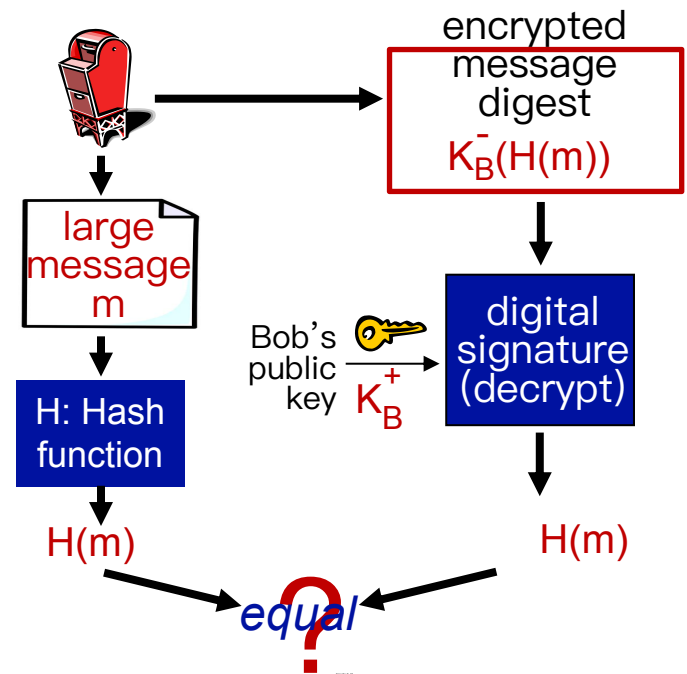
*different messages
but identical checksums!*

Digital signature = signed message digest

Bob sends digitally signed message:



Alice verifies signature, integrity of digitally signed message:



Hash function algorithms

- MD5 hash function widely used (RFC 1321)
 - computes 128-bit message digest in 4-step process.
 - arbitrary 128-bit string x , appears difficult to construct msg m whose MD5 hash is equal to x
- SHA-1 is also used
 - US standard [NIST, FIPS PUB 180-1]
 - 160-bit message digest

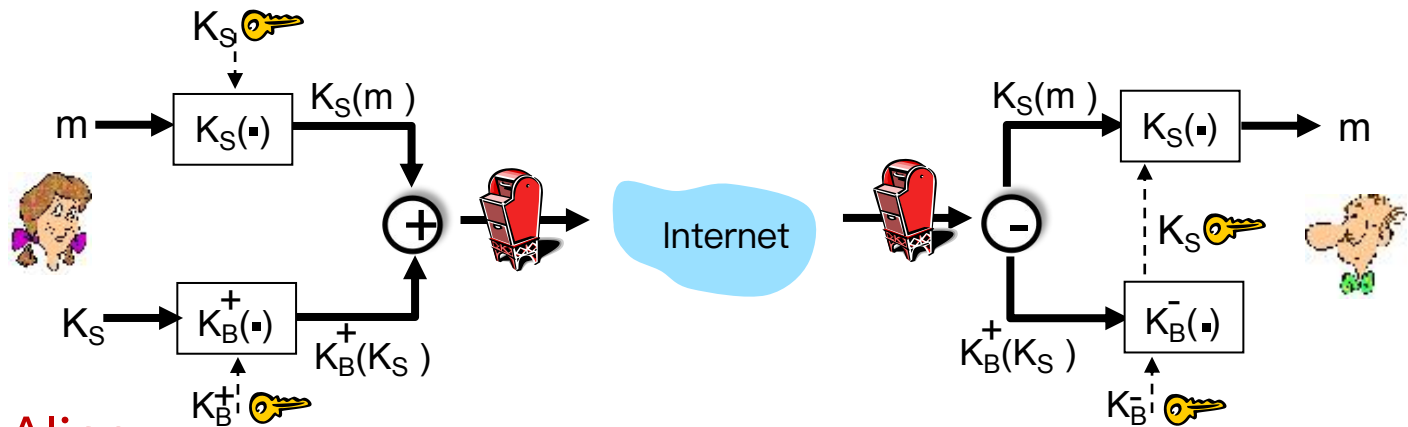
Chapter 8 outline

- What is network security?
- Principles of cryptography
- Authentication, message integrity
- **Securing e-mail**
- Securing TCP connections: TLS
- Network layer security: IPsec
- Security in wireless and mobile networks
- Operational security: firewalls and IDS



Secure e-mail: confidentiality

Alice wants to send *confidential* e-mail, m , to Bob.



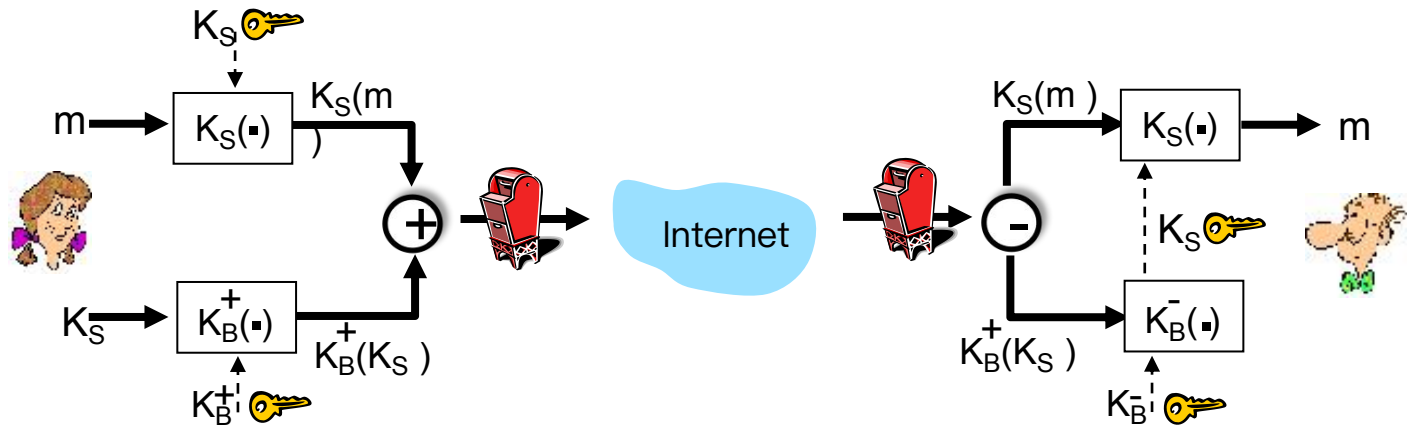
Alice:

- generates random *symmetric* private key, K_S
- encrypts message with K_S (for efficiency)
- also encrypts K_S with Bob's public key
- sends both $K_S(m)$ and $K_B^+(K_S)$ to Bob

Secure e-mail: confidentiality

(more)

Alice wants to send *confidential* e-mail, m , to Bob.

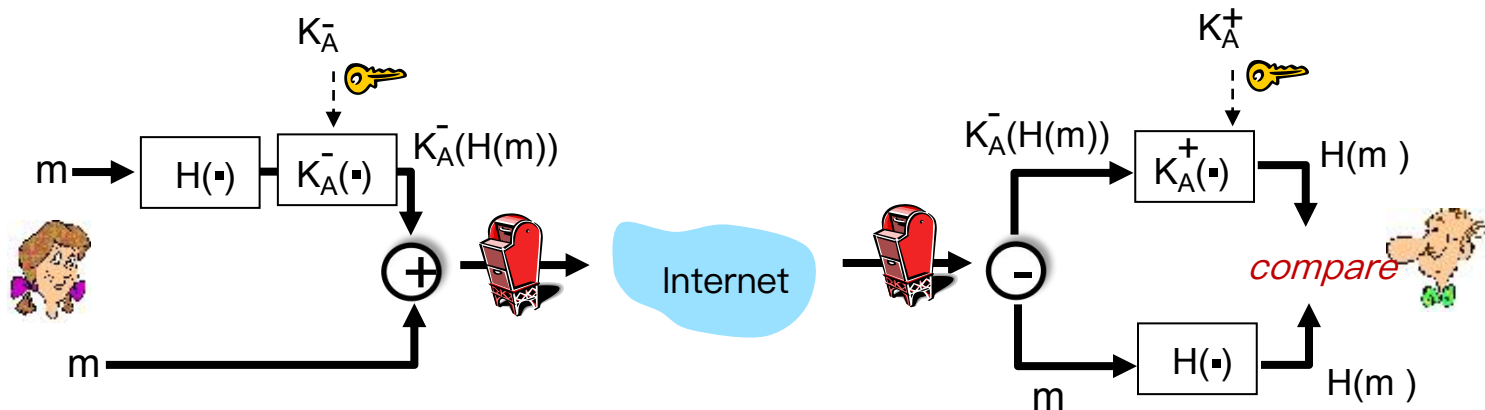


Bob:

- uses his private key to decrypt and recover K_S
- uses K_S to decrypt $K_S(m)$ to recover m

Secure e-mail: integrity, authentication

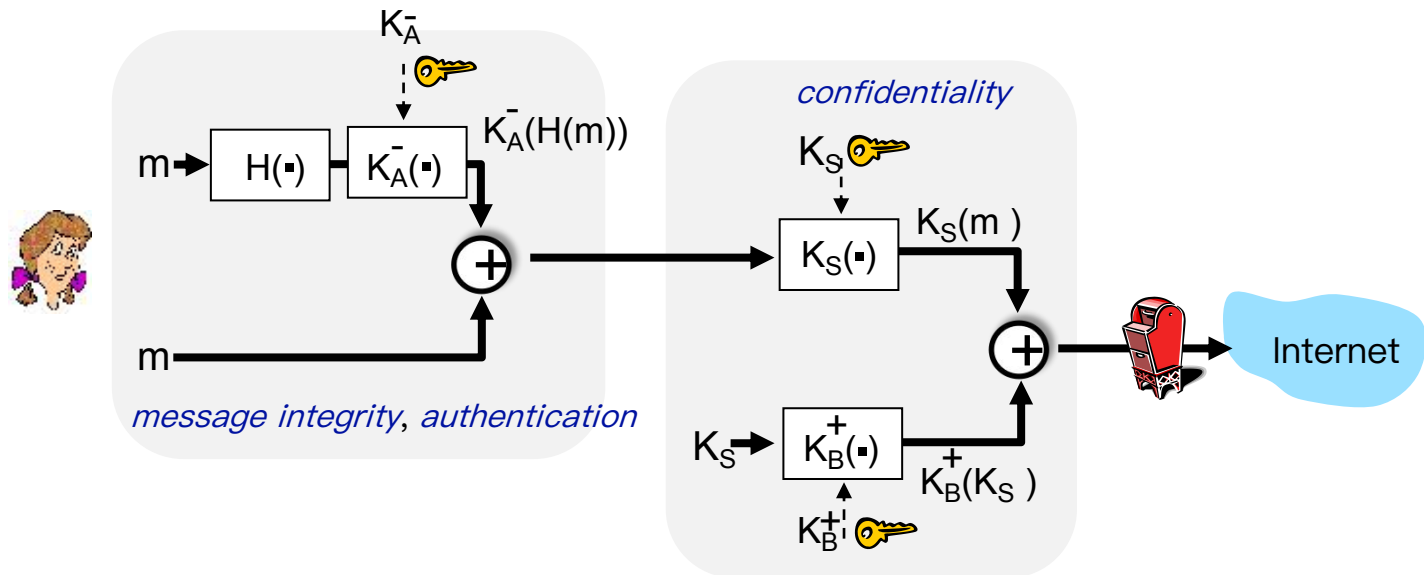
Alice wants to send m to Bob, with *message integrity, authentication*



- Alice digitally signs hash of her message with her private key, providing integrity and authentication
- sends both message (in the clear) and digital signature

Secure e-mail: integrity, authentication

Alice sends m to Bob, with
confidentiality, message integrity, authentication



Alice uses three keys: her private key, Bob's public key, new symmetric key

What are Bob's complementary actions?

Chapter 8 outline

- What is network security?
- Principles of cryptography
- Authentication, message integrity
- Securing e-mail
- **Securing TCP connections: TLS**
- Network layer security: IPsec
- Security in wireless and mobile networks
- Operational security: firewalls and IDS



Transport-layer security (TLS)

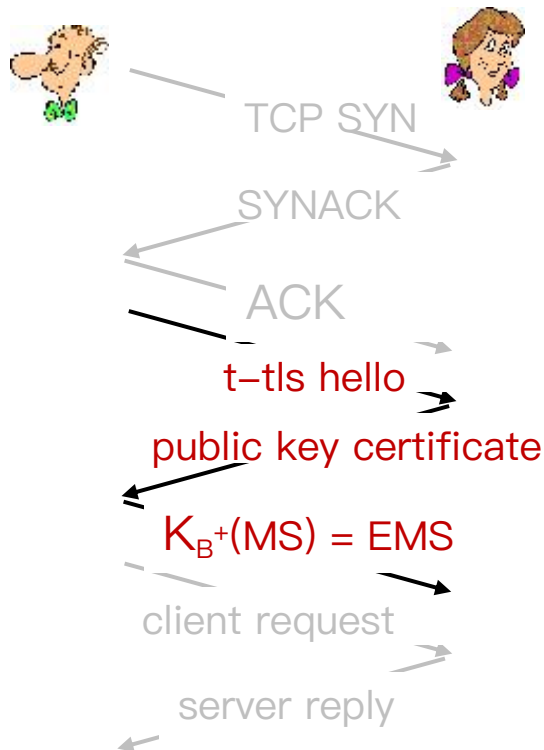
- widely deployed security protocol above the transport layer
 - supported by almost all browsers, web servers: https (port 443)
- provides:
 - **confidentiality**: via *symmetric encryption*
 - **integrity**: via *cryptographic hashing*
 - **authentication**: via *public key cryptography*

} *all techniques we have studied!*
- history:
 - early research, implementation: secure network programming, secure sockets
 - secure socket layer (SSL) deprecated [2015]
 - TLS 1.3: RFC 8846 [2018]

Transport-layer security: what's needed?

- let's *build* a toy TLS protocol, *t-tls*, to see what's needed!
- we've seen the “pieces” already:
 - **handshake**: Alice, Bob use their certificates, private keys to authenticate each other, exchange or create shared secret
 - **key derivation**: Alice, Bob use shared secret to derive set of keys
 - **data transfer**: stream data transfer: data as a series of records
 - not just one-time transactions
 - **connection closure**: special messages to securely close connection

t-tls: initial handshake



t-tls handshake phase:

- Bob establishes TCP connection with Alice
- Bob verifies that Alice is really Alice
- Bob sends Alice a master secret key (MS), used to generate all other keys for TLS session
- potential issues:
 - 3 RTT before client can start receiving data (including TCP handshake)

t-tls: cryptographic keys

- considered bad to use same key for more than one cryptographic function
 - different keys for message authentication code (MAC) and encryption
- four keys:
 -  K_c : encryption key for data sent from client to server
 -  M_c : MAC key for data sent from client to server
 -  K_s : encryption key for data sent from server to client
 -  M_s : MAC key for data sent from server to client
- keys derived from key derivation function (KDF)
 - takes master secret and (possibly) some additional random data to create new keys

t-tls: encrypting data

- recall: TCP provides data *byte stream* abstraction
- Q: can we encrypt data in-stream as written into TCP socket?
 - A: where would MAC go? If at end, no message integrity until all data received and connection closed!
 - solution: break stream in series of “records”
 - each client-to-server record carries a MAC, created using M_c
 - receiver can act on each record as it arrives
- t-tls record encrypted using symmetric key, K_c , passed to TCP:

K_c (*length* | *data* | *MAC*)

t-tls: encrypting data (more)

- possible attacks on data stream?
 - *re-ordering*: man-in middle intercepts TCP segments and reorders (manipulating sequence #s in unencrypted TCP header)
 - *replay*
- solutions:
 - use TLS sequence numbers (data, TLS-seq-# incorporated into MAC)

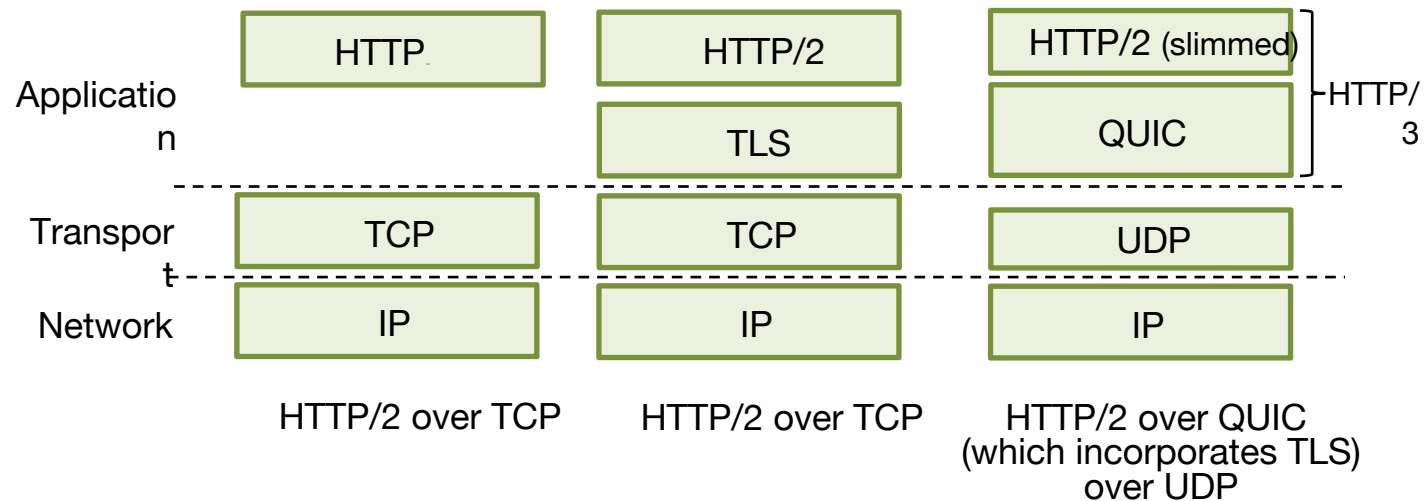
t-tls: connection close

- truncation attack:
 - attacker forges TCP connection close segment
 - one or both sides thinks there is less data than there actually is
- **solution:** record types, with one type for closure
 - type 0 for data; type 1 for close
- MAC now computed using data, type, sequence #

K_C (*length* | *type* | *data* | *MAC*)

Transport-layer security (TLS)

- TLS provides an API that *any* application can use
- an HTTP view of TLS:



TLS: 1.3 cipher suite

- “cipher suite”: algorithms that can be used for key generation, encryption, MAC, digital signature
- TLS: 1.3 (2018): more limited cipher suite choice than TLS 1.2 (2008)
 - only 5 choices, rather than 37 choices
 - *requires* Diffie–Hellman (DH) for key exchange, rather than DH or RSA
 - combined encryption and authentication algorithm (“authenticated encryption”) for data rather than serial encryption, authentication
 - 4 based on AES
 - HMAC uses SHA (256 or 284) cryptographic hash function

